Studying Task Processes for Improving Programmer Productivity

Pankaj Jalote, *Fellow, IEEE* , and Damodaram Kamma

Abstract—Productivity of a software development organization can be enhanced by improving the software process, using better tools/technology, and enhancing the productivity of programmers. This work focuses on improving programmer productivity by studying the process used by a programmer for executing an assigned task, which we call the task process. We propose a general framework for studying the impact of task processes on programmer productivity and also the impact of transferring task processes of high-productivity programmers to average-productivity peers. We applied the framework to a few live projects in Robert Bosch Engineering and Business Solutions Limited, a CMMI Level 5 company. In each project, we identified two groups of programmers: high-productivity and average-productivity programmers. We requested each programmer to video capture their computer screen while executing his/her assigned tasks. We then analyzed these task videos to extract the task processes and then used them to identify the differences between the task processes used by the two groups. Some key differences were found between the task processes, which could account for the difference in productivities of the two groups. Similarities between the task processes were also analyzed quantitatively by modeling each task process as a Markov chain. We found that programmers from the same group used similar task processes, but the task processes of the two groups differed considerably. The task processes of high-productivity programmers were transferred to the average-productivity programmers by training them on the key steps missing in their process but commonly present in the work of their high-productivity peers. A substantial productivity gain was found in the average-productivity programmers as a result of this transfer. The study shows that task processes of programmers impact their productivity, and it is possible to improve the productivity of average-productivity programmers by transferring task processes from high-productivity programmers to them.

Index Terms—Programmer, productivity, task processes, task execution, industrial study, software, efficiency, improvement, experiments

1 Introduction

Software productivity improvement can be achieved by improving the software process, technologies employed, and capability of people involved. During the last three decades, improving software process for better software productivity has been emphasized [1]. Improvements in the overall software process increase software productivity by identifying and eliminating waste during software development and optimizing existing methods to reduce the software development effort. Frameworks such as CMMI [2], ISO [3], and ASPICE [4] have emerged to help organizations improve their processes. Tools and technology have also evolved continuously to improve productivity.

Considerable work done on programmer productivity includes identifying factors that affect productivity, differences between novice and experts, how programmers spend time, pair programming, Personal Software Process (PSP), and so forth. Little work has been done to understand how programmers execute tasks assigned to them and how the process they use for executing the tasks may affect their productivity, which is the focus of this work.

In a software project, the project manager typically maintains a detailed schedule that specifies various project activities and to whom these activities are assigned. Though some activities (e.g., reviews) are performed by a group, most of these activities (e.g., program a module X, write test plan, test module Y) are assigned to one person [5], [6]. We use the term *task* to refer to an activity in the software project that is assigned to one person (sometimes called a resource) and which has a clear deliverable and a completion condition that is used by the project manager to assess its satisfactory completion. The *task process* of a programmer for a task is the sequence of steps the programmer uses to execute the assigned task.

It is known that some programmers are much more productive than others, despite the same level of experience [7], [8]. We hypothesized that task processes used by programmers for performing assigned tasks may have an impact on programmers productivity.

This work aimed to conduct a systematic study of task processes programmers use to execute assigned tasks and how it may affect their productivity. The overall software process generally does not standardize any particular task process, and hence a task process for a task may vary from one programmer to another. Another important goal of this work was to study whether the productivity of programmers can be improved by modifying their task processes.

- *P. Jalote is with the IIIT-Delhi, New Delhi, Delhi 110020, India. E-mail: jalote@iiitd.ac.in.*
- *D. Kamma is with the Robert Bosch Engineering and Business Solutions Limited, Bengaluru, Karnataka 560095, India. E-mail: damodaram.kamma@in.bosch.com.*

Manuscript received 17 Jan. 2018; revised 18 Feb. 2019; accepted 23 Feb. 2019. Date of publication 11 Mar. 2019; date of current version 16 Apr. 2021. (Corresponding author: Damodaram Kamma.) Recommended for acceptance by A. Begel. Digital Object Identifier no. 10.1109/TSE.2019.2904230

0098-5589 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

The main research questions this paper aims to address are as follows:

RQ1: Do task processes used by programmers affect their productivity?

RQ2: Do programmers use a similar task process while executing a similar type of tasks?

RQ3: How similar are task processes of high- and average-productivity programmers?

RQ4: Can the productivity of average-productivity programmers be improved by transferring task processes of high-productivity programmers to them?

The idea of understanding RQ1 is to identify the basis of exploring the other research questions. RQ2 focuses on understanding whether programmers use similar task processes for similar tasks and whether the level of similarity is different for high- and average-productivity programmers. RQ3 focuses on comparing the task processes of programmers within each group (high- or average-productivity group) and across groups to understand whether programmers in a group use similar task processes and whether the task processes of the two groups are different. If differences are observed between the task processes of the two groups, RQ4 is the natural question.

To understand the task processes, we captured the videos of the computer screens of the programmers while they were executing the tasks and then extracted the steps executed and time spent on different steps by them. Using these steps, we defined the task process. For the initial study, the task process was represented using a table. For quantitatively analyzing the differences between the task processes, we represented a task process as a Markov chain and used the distance between the two Markov chains to represent the difference between the task processes. These are explained later in the paper.

Three different field studies were conducted for 2.5 years to address these research questions. All the studies were on model-based software projects and focused on programmers with 1—3 years of experience, executing unit-testing tasks. In the first study, we analyzed 14 task processes of 6 programmers in 1 project, with at least 2 tasks by each programmer, to address RQ1. In the second study, we analyzed 36 task processes of 18 programmers from 3 live projects—project 1 had 6 programmers (3 each of high and average productivity), project 2 had 5 programmers (3 with high productivity and 2 with average productivity), and project 3 had 7 programmers (4 with high productivity and 3 with average productivity)—to look at the similarity between task processes (i.e., RQ2 and RQ3). The study for the impact of transferring task processes of high-productivity programmers to average-productivity programmers (i.e., RQ4) was conducted on 24 programmers from 3 live projects—project 1 had 12 programmers (5 with high productivity and 7 with average productivity) and projects 2 and 3 had 6 programmers each (3 each with high and average productivity).

The rest of the paper is organized as follows: Section 2 presents the related work. Section 3 discusses how a buy-in is obtained to conduct this study on live projects in Robert Bosch Engineering and Business Solutions Limited. Section 4 explains the task and task processes. Section 5 presents the framework for studying the impact of task processes on programmers' productivity. The subsequent three sections report three field studies conducted on live model-based unit-testing projects to address the research questions. Section 9 states the limitations and threats to the validity of our work, and Section 10 concludes the study.

2 Related Work

Different aspects of the productivity of programmers have been evaluated. Detailed factors that affect software productivity were identified on the basis of surveys, interviews, and observation of programmers and project managers. Some of them include programmer's experience [9], work environment [10], complexity of programs [11], ambiguity in programmer's role [12], reuse of software artifacts [13], accuracy of requirements [14], communication between team members [15], goals set for programmers [16], use of tools and automation [17], team size [18], task switching's and interruptions [19], adoption of process frameworks [20], and attrition of team members [21].

PSP, defined by Humphrey, focuses on improving individual programmer productivity [22]. PSP uses a systematic process (with checklists, forms, templates, etc.) to analyze one's own capabilities, based on the past data, and make plans for improving quality and reducing schedule deviation [23]. PSP helps programmers minimize defects in their tasks and improve productivity [22], and improve effort estimation accuracy and schedule variation [24]. PSP follows a self-learning approach rather than learning from others or peers, and requires a lot of adaptations to match company practices for its effective implementation in organizations [23], [25].

Pair programming can also impact programmer productivity. Programmers work in pairs to develop software or solve an issue/problem/bug. Continuous reviews and discussions between them trigger fast learning and help improve programmers' productivity. However, there is no consensus that pair programming improves programmers' productivity [26], [27].

In software development, peer reviews [28], [29] and walk-throughs [30] have been widely used for a long time and recognized as a best practice. These focus on examining the work products developed by programmers to identify defects and improve their quality, but they do not look at the process used by programmers for developing the products, which is the focus of this work.

Many studies focused on how programmers comprehend software and also on identifying differences between novices and experts [31], [32]. Some studies looked into how programmers spent time at office on work- and non-work-related activities [33]. A few studies focused on identifying how programmers perform debugging, refactoring a piece of code, task/activity switching, and so forth [34]. Hartmann et al. first observed how expert programmers fixed their compilation errors and then integrated the same in the development environment of novice programmers for improving their productivity [35]. Robillard

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Questionnaire for the Survey of Project Managers

S. No	Questionnaire	Yes	No
1.	Do associates in your team break a task into a sequence of sub-tasks (called task process)?	91.9%	8.1%
2.	Do your associates follow various task-processes in executing tasks?	76.9%	23.1%
3.	Do task processes followed by an associate in his/her tasks affect his/her productivity?	84.6%	15.4%
4.	Can studying the way an associate breaks a task into a sequence of sub-tasks help in identifying the areas of improvement?	91.4%	8.6%
5.	Can studying the task processes followed by associates in their tasks can help in identifying best practices for executing tasks?	84.8%	15.2%
6.	Which tasks consume maximum effort in your project?	-	-

observed five programmers for about 2 hours while they investigated and fixed a bug in a given piece of code and found that programmers comprehended programs differently [36]. Bowdidge observed six pairs of programmers and found that two pairs of programmers followed a well-planned systematic approach while the remaining four pairs mostly used an exploratory approach [37]. Lawrence observed 12 programmers when they were fixing their assigned bugs. He found that programmers predominantly required information for understanding and fixing bugs. He suggested tool designers to find out various ways of providing information to programmers [38].

Studies have also shown that a few programmers are more productive than others. Grant and Sackman [7] observed 12 programmers debug a given task and found that the time taken by the fastest programmer to the slowest programmer was in the ratio of 28:1. Similar results were also observed in the study conducted by Prechelt [8].

The notion of improving the productivity of people by studying other workers started in industrial engineering at the end of the nineteenth century. Taylor observed the execution of mechanical tasks by each worker, and these observations that were based on the time-motion study were developed into a theory called Taylorism—also known as scientific management theory [39]. This theory helped many organizations improve productivity [40]. Taylorism states that a few individuals are more talented than others, and organizations should observe these talented individuals to uncover the best ways of performing a task for training people.

3 Getting the Buy-In

Due to inherent uncertainties involved in exploring research questions, it is generally difficult for researchers to convince business organizations for conducting empirical studies to address some research questions. The situation gets further complicated in software services companies where projects are executed on tight deadlines and budgets.

Programmers and project teams are also often not interested in participating in these studies, as it will necessarily involve some extra work, and the teams are always under pressure, and their performance goals generally do not include support for field studies. Consequently, empirical studies of actual practices are hard to conduct [41], and studies that require direct observation of programmers are few [42].

We realized that for doing an empirical study of task processes on live projects, it is extremely important to first establish its desirability and motivation and engage stakeholders in the design of the study. This will not only provide inputs but also make them a part of the study, which will further help in getting the necessary support and commitment.

To get inputs about the value of the study as well as their buy-in, we conducted a Web-based survey of project managers to find out their views on studying task processes as a possible approach for improving programmer productivity. The survey was sent to 189 project managers, out of which 115 (more than 60 percent) responded. Table 1 presents some of the key questions of the questionnaire and their responses.

This survey clearly indicated that project managers felt that the task processes used by a programmer (called associate internally) impacted their productivity, and we could uncover best practices and improve the productivity of programmers by studying task processes. We also got useful inputs on the main tasks in a project where programmers spent the maximum effort, which helped identify tasks the research should focus on.

We presented our research proposal along with the survey results to a few key senior managers, who advised us to focus on model-based projects because the majority of their projects were expected to be in model-based development in the future. As many programmers were involved in unit testing, this task was selected for the initial study. Further, we were advised to consider programmers with less than 3 years of experience, as they were the major workforce in the organization. We then met with the respective senior and middle-level managers for nominations of projects for our research.

We spent more than 6 months in this process. We feel that for such studies, researchers should be adequately prepared for investing the effort needed to get a buy-in for conducting an empirical study in an organization.

4 Task and Task Processes

Most software projects executed in commercial organizations have an overall plan that specifies the broad schedule with major milestones and is often obtained by estimating the effort and time for the project and major components. This overall project plan is essential for scheduling and monitoring the project, but in terms of execution and

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

control, it is of limited use for a project manager. For a finer-level control, this overall plan is generally converted by a project manager into a detailed schedule, which is in line with the overall schedule. The detailed schedule of a project, which typically evolves during project execution, has several activities, many of which are assigned to one person. We refer to such activities that are assigned to one person as *tasks*.

In most projects, a task may take a few hours to a few days to complete. Typically, a programmer would execute an assigned task incrementally in small *steps*, with each step involving some well-defined work that would help in the completion of the task and often result in some tangible output that the programmer uses in later steps. How the execution of these steps is organized by a programmer is what we refer to as *task process*. In other words, a task process for a task is the sequence of steps used by a programmer for executing that task.

As the overall software process does not specify or standardize task processes, execution of a task may vary from one programmer to another. For example, for a task "develop functionality of a module X," a programmer may use the following task process (specified as the sequence of steps): build a skeleton, create basic functions, define methods and attributes, write code for a method, unit test the method, and repeat the last two steps as many times as required. Another programmer may use the following task process for the aforementioned task: write a small piece of code, compile, fix compiling errors, repeat the steps till code for module X is done, test the entire code, fix uncovered bugs, and repeat testing and debugging till satisfied.

Besides the structure of task processes in terms of steps employed and their order, the execution of a step can also vary from one programmer to another. For example, a programmer may write a few lines and then compile, while another may write 10 or 100 s of lines before compiling. Similarly, in the testing step, a programmer may execute one or two test cases, and another may execute 10 s of test cases.

A task process can be represented as a table containing the sequence of steps executed by a programmer while executing a task. For each step in the table, we recorded the time spent on the step as well as other relevant information (e.g., how the step was performed). As a task is eventually completed, this table has a limited number of steps, with some steps repeated many times in this sequence. Consequently, the number of distinct steps (or step types), represented by the name given to the step, is normally much lower than the number of steps performed in the task process.

Some of the features of different task processes can be easily determined by comparing their respective tables. For example, the tabular representation is sufficient to determine whether there are different steps in two task processes; however, the same does not suffice to quantitatively understand how similar or different the two task processes are. For computing the similarity between two processes, we represent the process as a Markov chain, where each step is a node, and movement from one step to another is a transition. This representation is useful to quantify the difference between two task processes—the distance between the two Markov chains can be used as a measure of how similar/dissimilar the two task processes are. We discuss this later in the next section.

5 Approach for Studying the Impact of Task Processes on Programmer Productivity

For studying the impact of task processes on the productivity of programmers, we first identified some projects of reasonable duration and that involved multiple programmers. We then identified two groups of programmers in each project—high-productivity programmers and average-productivity programmers. We then video captured their screens while they performed the tasks and from the videos defined the steps and the task processes being used.

Both the programmer groups were briefed about the study and their involvement, particularly in capturing data about the tasks. The programmers were assured that the data collected would be used only for the study, and all information about their tasks would be shared and verified with them. We further assured them that the details about their tasks or productivity would not be revealed to the management and that the management had already agreed to that. Based on the advice of senior managers, we kept them uninformed about classifying programmers into high- and average-productivity groups but informed them about the intention of studying the executions of tasks by programmers to uncover the best ways of executing a task. This helped avoid conflicts and biases. However, we shared our full approach with the senior management and project managers.

5.1 Identifying Projects and Tasks for Studying

It is important to choose projects of similar type so as to minimize the impact of project type on the study. For this study, projects were considered of similar type if they were in the same domain and used similar platform, language, overall processes, tools, development environment, and so forth. We focused on model-based software projects for the study, as this was of interest to the organization, as discussed earlier.

A model-based development differs somewhat from the traditional software development. Requirements are generally addressed in a similar manner. However, in place of high-level design, generally a meta-model is developed, and in place of detailed design, a detailed model is built for a unit/component, generally using a formal language. The detailed model is used to generate code, which is executed for unit testing of the detailed model. Unit testing is an activity that consumes a considerable amount of resources, and in Bosch it is done by a different team than the team that develops the detailed model. In out study, we have concentrated on unit testing tasks.

We selected model-based projects that had a reasonable number of programmers involved in unit testing and a stable project manager. In each project, we selected multiple programmers with high and average productivity and also at least two tasks of each group of programmers for which detailed data were collected.

The project manager and programmers of the selected projects were first briefed about this study for a common understanding and their support and buy-in. For studying

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

task processes, programmers were trained in aspects such as self-recording videos. They were also engaged in the review of the project at all stages and the information we collected on tasks or projects.

5.2 Identifying High- and Average-Productivity Programmers

Our approach required identifying high- and average-productivity programmers in a project. In software, *productivity* is defined as the ratio of software size to the total effort spent on developing the software [43], [44], [45]. Studies have used various size measures for measuring programmer productivity, such as function points per hour [46], lines of code per hour [47], number of tasks per month [48], number of modified requests per year [49], number of use cases per effort [50], number of testable requirements per unit effort [51], and so forth.

For a task, the *size* was measured as Adjusted Testable Requirements (ATRs), a measure that the organization has been using for the past 9 years. Discussion with programmers, project managers, and quality department showed that the size measure was stable for unit-testing tasks, and even customers accepted that. ATR is a combination of testable requirements and function points. It requires first splitting the requirements to low-level requirements such that each low-level requirement is testable. Then, the complexity of each requirement is measured using parameters required for each requirement, such as number of interfaces and memory accesses. Finally, ATR is computed as a function of all weighted low-level requirements where the weight of each low-level requirement represents the complexity of the respective requirement. The organization provides regular training on this size measure to project managers, who record the size as a part of their planning and scheduling. We did not introduce any new size measure but used the ATR size measure for computing productivity of programmers due to its wide acceptability within the organization and the confidence of people in it.

We determined the *effort* spent by a programmer for executing his/her assigned tasks from their task videos. We also verified the effort captured from the videos with the respective programmer. *Programmer productivity* for a unit-testing task is computed as a ratio of the size of assigned task in ATR to the total effort spent in hours on executing the task.

As the task-level past productivity data of programmers was not available for grouping programmers into high-productivity and average-productivity programmers, we first requested project managers to rate programmers on productivity as high or average. However, we later determined the productivity of each programmer from the data of our study and compared it with the subjective evaluation of project managers for grouping programmers into high- and average- productivity programmers. The initial assessment of project managers turned out to be accurate.

5.3 Capturing a Programmer's Task Processes

For studying task processes, we need to capture the effort data of programmers at a finer granularity and with higher confidence about accuracy than what is achieved by effort and commonly used activity logging systems. For example, the standard method of the organization is to have programmers submit a weekly activity report in which they report the effort spent on different tasks assigned to them.

To obtain data at a much finer granularity for analyzing the task processes effectively, we used the tool "snag-it" [52] to capture the screenshots of programmers' monitors while they were executing the tasks. Before installing the tool, the programmers were assured that the captured videos would not be shared with anyone and were meant only for research purpose. Actual data collection for the experiment started only after a programmer became familiar with the capturing of screenshots using the tool.

The tool captured one screenshot of the computer monitor every 5 seconds of task progression. As the study was performed in close coordination with programmers and the data from videos were always verified with programmers, we felt that capturing the screen every 5 seconds was sufficient. This sequence of screenshots was converted into a video in which one could see the screen of a programmer moving at a fast speed, without actually watching the programmer. In the replay of the screens also, we did not notice discontinuities or gaps.

To identify steps in task processes of programmers and how they are organized from the video, we first identified the commonly used steps for executing a task based on discussions with the project teams. Generally a common vocabulary exists that programmers use to discuss how they execute a task, which, with some refinements and standardization, is a good starting point for the set of steps that may exist in a task process. A general task process also exists for key tasks, which are used in training programs.

We first manually analyzed a task video of a programmer to identify the steps, their properties, and their execution sequence. This required a huge amount of effort, but we could not find any other way of getting the data at the level of accuracy that we needed for this study. We generally grouped what appeared as a continuous activity over many minutes as one step and used visible transitions to a different type of continuous activity as a transition to the next step. We noted these steps in a table, along with their duration and some notes. (Where needed, we later suitably changed the name to align with the names of the steps initially identified.) We discussed this analysis with the programmer to validate and refine it based on his/her inputs. During refinement, steps may get merged into one, or a step may get split into separate steps—if this happened, we had to go back and reanalyze the video to determine the durations. While this required subjective judgment for identifying the steps, we felt that we were able to capture the steps reasonably correctly by validating these with the programmers.

We finally captured the task process as a table containing the sequence of steps executed, time spent in each step, and additional information about the steps. Obtaining this table representing the task process from the video is an iterative process. Generally, in the first instance, a table is prepared from the video, in which different phrases and terms are used to express the different steps, and some information may also be captured while noting down the step. From this initial table prepared from the video, a new table is

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

extracted in which a standard set of names of steps is used and information about the step is separated out. Finally, based on inputs from the programmers, the table may be suitably modified.

Although the starting and end points of task processes of different programmers for a task are generally the same, task processes of different programmers progress in a different manner. Our initial study and interactions with programmers indicated that the differences in their task processes may lie not just in the nature of steps but also in the manner how they were executed. Therefore, besides identifying the steps and their organization, we also noted some qualitative information about how each step was executed. For example, for the step "write new test cases," the attributes include the number of test cases after finishing each iteration, time taken in each iteration, how test cases have been derived, total number of test cases, and total time taken. Similarly, for the step "setup test project," the attributes include time spent in an iteration, what has been modified/added, overall time spent in this step, and so forth.

5.4 Modeling the Task Process as a Markov Chain

The tabular representation is insufficient for studying the similarity between task processes quantitatively. Therefore, we represented a task process as a Markov chain and used the distance between two Markov chains to capture the difference between task processes.

A good amount of work has been done on modeling and analyzing business processes [53], [54]. Process models defined in the literature [55], [56], [57], [58], [59], [60], [61] focus on representing the overall software process. Petrinets were used to model business process tasks and recover complex dependencies between them through a formal analysis [62]. Most of these approaches are used for modeling the overall software process and not steps in a task. These approaches are suitable for capturing the process and not for determining quantitatively how similar two processes are.

We used Markov chains to model the task process of a programmer. Markov chains are used in different domains on numerous applications for modeling and analyzing a stochastic (random) process [63]. Typically, a stochastic process representing a system moves randomly between various system states over time. Markov chains model the evolution of such stochastic processes using system states and transitions between the states. As a programmer's next step generally depends on the outcome of the present step, a criterion for applying Markov chains is satisfied.

A task process of a task executed by a programmer corresponds to one Markov chain. In a Markov chain model of a process, a state represents a distinct condition, and transition probabilities represent the probability of reaching from one state to another during the execution of the process. For the Markov chain of a task process, the unique steps are the states. If the programmer executed a step *j* after step *i* in the task process, we added a transition from state *i* to state *j*. The probability of transition from state *i* to state *j* is the ratio of the number of times in the task process the programmer goes from step *i* to step *j* to the total number of times the programmer goes from step *i* to any other step. The Markov chain can also be represented as a transition matrix, with an entry (*i*,*j*) in the matrix giving the probability of transition from state *i* to state *j*. Examples of Markov chains for task processes are given later.

5.5 Difference Between Task Processes as Distance Between Markov Chains

With the Markov model of a task process, we can model the difference between two task processes as the distance between the Markov chains of those task processes. The distance between two Markov chains can be computed easily by their state transition matrices.

The distance between two Markov chains can be measured in many ways. We used the sum of absolute differences or Manhattan distance in this study [64]. Suppose A and B are two transition matrices of size $n * m$ representing two Markov chains. $A_{ij}(B_{ij})$ is the probability of transitioning from state *i* to state *j* in matrix A(B). The Manhattan distance between A and B is

$$L_1 = \sum_{i=1}^{n} \sum_{j=1}^{m} |A_{ij} - B_{ij}|.$$

Two task processes are exactly the same when the Manhattan distance between their state transition matrices is zero, that is, both the state transition matrices are the same. The dissimilarity between two task processes increases as the Manhattan distance between their state transition matrices increases.

5.6 Analyzing the Impact on Productivity

To study the impact of task processes on programmer productivity, we first categorized task processes into two groups: task processes of all high-productivity programmers and task processes of all average-productivity programmers. From these task tables, we obtained a consolidated table for the average- and high-productivity tasks capturing a summary of how high- and average-productivity programmers used the steps in their task processes. From these tables, we identified key differences between the task processes of the two groups. We verified our observations with the project managers and programmers. Subsequently, we studied the impact of these steps on the productivity of the two groups.

We also transferred the task processes of high-productivity programmers to average-productivity programmers by training the average-productivity programmers to use high-productivity task processes. To study the impact of this transfer, we again collected task videos for new tasks from average-productivity programmers to study their new task processes and productivity. We compared the productivity achieved while using old and new task processes by average-productivity programmers. We also quantitatively studied how the similarity between the task processes of average-productivity programmers and high-productivity programmers changed.

6 Analyzing the Impact of Task Processes on Programmer Productivity

The goal of the first field study was to understand whether high-productivity programmers execute their tasks differently when compared with the execution of similar tasks by

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

TABLE 2
Task Process (TP-H) of a Task Executed by High-Productivity Programmer

Duration	Steps
0:10	Analyze task request
0:20	Analyze functional document
0:15	Load files
0:40	Setup test environment
0:01	Analyze functional document
0:06	Create/modify aspects.
0:05	Analyze functional document
0:15	Copy information to excel
0:02	Automation (excel formulas) for output variable values
0:05	Copy information to excel
0:05	Analyze functional document
0:05	Write test case inputs
0:02	Automation (excel formulas) for output variable values
0:30	Write test case inputs
0:05	Run test cases
0:04	Understand code behavior
0:02	Write test case inputs
0:02	Run test cases
0:05	Understand code behavior
0:02	Automation (excel formulas) for output variable values
0:01	Analyze functional document
0:15	Write test case inputs
0:02	Run test cases
0:01	Understand code behavior
0:06	Write test case inputs
0:02	Run test cases
0:02	Understand code behavior
0:02	Analyze test results
0:08	Analyze code coverage
0:02	Modify test case inputs
0:10	Write test case inputs
0:01	Run test cases
0:01	Analyze test results
0:08	Analyze code coverage
0:40	Documentation
0:40	Update Review work/checklists
0:01	Run test cases
0:08	Analyze code coverage
0:05	Update Review work/checklists
0:10	Archive
0:05	Close task

average-productivity programmers (RQ1). This study was conducted on six programmers, three of whom were rated as high productivity and the remaining three as average productivity in one project. A total of 14 unit-testing tasks were captured, 7 by each of the 2 groups, with each programmer capturing at least 2 tasks. Some of the results of this study were earlier published in [65].

6.1 Task Process of a Programmer

The task processes were identified as described earlier. Table 2 presents the task process for 1 task (TP-H) of a high-productivity programmer, indicating that the programmer executed 41 steps (of 17 distinct step types) and spent a total of 351 minutes (approximately 6 hours).

Table TP-A presents the task process for a similar task by an average-productivity programmer (see Table 3), indicating this task process had 89 steps with 17 distinct steps and a total of about 428 minutes were spent in executing the task. Although the starting and end points of these two are the same, the way these task processes progressed was different. Also, some differences existed in the manner different steps were executed.

6.2 Differences in Task Processes of Average- and High-Productivity Programmers

To identify the difference between task processes, we could just look at the task process tables and identify similarities and differences, as was done in our initial study [65]. To identify the differences between the task processes of average- and high-productivity programmers, we noted all the steps in the task processes, counted the number of task processes that used these steps, and determined averages of frequencies and times spent in the steps by the average- and high-productivity programmer groups. Table 4 shows the frequency and time spent on some of the key steps. In the table, the following terms are used:

- *#TP:* Number of task processes that used the step
- *Avg Freq:* Average number of times a step was executed in a task process (determined as the number of times used in all the task processes/total number of task processes)
- *Tot Time:* Average total time duration spent on the step of a task process (in minutes)

The table indicates that certain steps are used by all (or almost all) high-productivity programmers but not used at all (or used in very few tasks) by average-productivity programmers, for example, Copy information to Excel, Automation, Write test case inputs, and Modify test case inputs. Similarly, certain steps are widely used by average-productivity programmers but not used much by high-productivity programmers, for example, Write test cases, Modify oracles of test cases, and Modify test cases.

The table also shows that high-productivity programmers spend much less time in certain steps compared with average-productivity programmers, for example, Create/Modify aspects, Run test cases, and Understand code behavior.

These observations show that the task processes for the two groups of programmers differ in steps they execute as well as in how the steps are executed or how much time is spent on them. Overall, we identified the following key differences:

High-Productivity Programmers had Extra Steps Before Writing Test Cases. Almost all the high-productivity programmers had two steps that the average-productivity programmers did not have: Copy information to Excel and Automation (Excel formulas) for output variable values. In these steps, they wrote an Excel formula to determine the expected output for the test case inputs based on their analysis of the design document and requirements. This extra step validated the expected output, and hence they ended up spending much less time in the steps Modify oracles of test cases, Modify test cases, Write test cases, Run test cases, Understand code behavior, and so forth. On the contrary, an average-productivity programmer did not perform these steps and determined the expected output manually when writing the test cases, thereby spending more effort in many other steps.

Average-productivity programmers wrote more test cases compared with high-productivity programmers to achieve the same

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Task Process (TP-A) of a Task Executed by an Average-Productivity Programmer

Duration	Steps
0:05	Analyze task request
0:10	Load files
0:20	Analyze functional document
0:45	Setup test environment manually
0:10	Create/modify aspects.
0:10	Write test cases
0:10	Create/modify aspects.
0:01	Run test cases
0:02	Understand code behavior
0:01	Modify test case inputs in excel
0:01	Run test cases
0:03	Understand code behavior
0:01	Modify test case inputs in excel
0:02	Run test cases
0:02	Understand code behavior
0:10	Write test cases
0:02	Run test cases
0:01	Understand code behavior
0:01	Modify oracles of test cases
0:02	Run test cases
0:01	Understand code behavior
0:15	Write test cases
0:02	Run test cases
0:01	Understand code behavior
0:01	Run test cases
0:01	Understand code behavior
0:01	Modify oracles of test cases
0:02	Run test cases
0:01	Understand code behavior
0:01	Modify oracles of test cases
0:02	Run test cases
0:01	Understand code behavior
0:10	Write test cases
0:02	Run test cases
0:01	Understand code behavior
0:15	Write test cases
0:02	Run test cases
0:01	Understand code behavior
0:05	Modify test case inputs in excel
0:02	Run test cases
0:01	Understand code behavior
0:01	Modify oracles of test cases
0:01	Run test cases
0:01	Understand code behavior
0:15	Write test cases
0:02	Run test cases
0:01	Understand code behavior
0:01	Modify test case inputs in excel
0:02	Run test cases
0:01	Understand code behavior
0:01	Modify oracles of test cases
0:02	Run test cases
0:01	Understand code behavior
0:01	Modify oracles of test cases
0:02	Run test cases
0:01	Understand code behavior
0:05	Write test cases
0:02	Run test cases
0:01	Understand code behavior
0:05	Write test cases
0:02	Run test cases
0:01	Understand code behavior
0:02	Modify test case inputs in excel
0:02	Run test cases
0:01	Understand code behavior
0:10	Derive oracles of test cases
0:01	Create/modify aspects
0:02	Run test cases
0:01	Understand code behavior
0:02	Create/modify aspects
0:02	Run test cases
0:01	Understand code behavior
0:10	Documentation
0:02	Analyze functional document
0:15	Documentation
0:03	Analyze test results
0:25	Documentation
0:08	Analyze code coverage
0:01	Modify test case inputs in excel
0:10	Write test cases
0:02	Run test cases
0:15	Update review checklists
0:01	Understand code behavior
0:03	Analyze test results
0:08	Analyze code coverage
0:10	Documentation
0:15	Update review checklists
0:15	Archive
0:05	Close task

code coverage set for the chosen project. The number of test cases written was obtained from the qualitative information captured. Further, *average-productivity programmers used more iterations of writing new test cases and running test cases followed by modifying test cases*.

During the project setup step, *all the average-productivity programmers loaded most of the library modules to test them, whereas the high-productivity programmers loaded only the required library modules into the test project setup*.

Although the average total time spent on the step "Analyzing functional document (or analyzing requirements)" is similar for both high- and average-productivity programmers in Table 4, the average frequency of visiting this step is lesser for the high-productivity programmers. This implies that *the high-productivity programmers spent most of their time to understand the requirements early in a few iterations*. This could also have an impact on the frequency and time spent on other steps.

There were some other differences also. Based on our understanding of the tasks, we felt that those were less significant for productivity improvement.

6.3 Impact of Process Differences on Productivity

Fig. 1 shows the software size (ATR) of assigned tasks to high- and average-productivity programmers. We did not see any significant difference between the sizes of tasks assigned to programmers in both the groups.

We determined the productivity of programmers of the two groups for the tasks we studied. The results are shown as a box plot in Fig. 2. As the data were normally distributed (as per Anderson—Darlington normality test), two-sample t tests showed that the project manager had rated their team members accurately on a productivity scale [high, average]

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

TABLE 4
Averages for the Task Processes of Average-Productivity and High-Productivity Programmers

Step Name	Average-productivity programmers			High-productivity programmers		
	Total task processes =7			Total task processes =7		
	#TP	Avg Freq	Tot Time	#TP	Avg Freq	Tot Time
Analyze task request	7	1	6.4	7	1	7.85
Analyze functional document	7	4.85	29.28	7	3.42	30.71
Load files	7	1.28	15.71	7	1	12.85
Setup test environment	7	3.71	47.14	7	1.28	37.85
Create/modify aspects.	7	3.71	27.57	7	1.14	13.42
Copy information to excel	0	0	0	7	1.42	21.85
Automation (excel formulas) for output variable values	1	0.28	1.5	6	2.14	14.57
Write test cases	6	6.28	67.15	1	0.85	8.14
Write test case inputs	1	1.42	11.42	6	4.85	55.14
Modify oracles of test cases	5	3.14	16.57	0	0	0
Derive oracles of test cases	2	0.428	2.85	1	0.14	0.71
Modify test cases	4	2.28	11.71	0	0	0
Modify test case inputs	3	1.71	5.15	7	1.71	9.45
Run test cases	7	15.85	35.4	7	7.14	15.14
Understand code behavior	7	15.14	45.71	7	6.42	24.57
Analyze test results	7	2.14	8.57	7	2.14	7.85
Analyze code coverage	7	2.42	22.14	7	2.42	22.4
Documentation	7	2.42	65	7	2.42	44
Update Review work/checklists	7	1.57	32.87	7	1.57	33.57
Archive	7	1	17.14	7	1	14.28
Close task	7	1	5.28	7	1	5.42

with a statistically significant value of 0.001. This analysis confirmed that the project manager's evaluation regarding productivity was consistent with the actual productivity data. It also showed that the average productivity of the two groups differed by a factor of two. (The data also sheds some light on the issue of reliability of a subjective evaluation by project managers about the technical competence of engineers working on the project.)

To check the impact of automation, the productivity of programmers was grouped according to the way they derived expected values; the same is shown in Fig. 3. As the data were normally distributed (as per the Anderson—Darlington normality test), two-sample t tests showed that the use of Excel formulas while generating test cases had a significant impact on programmer productivity with a P value of 0.073.

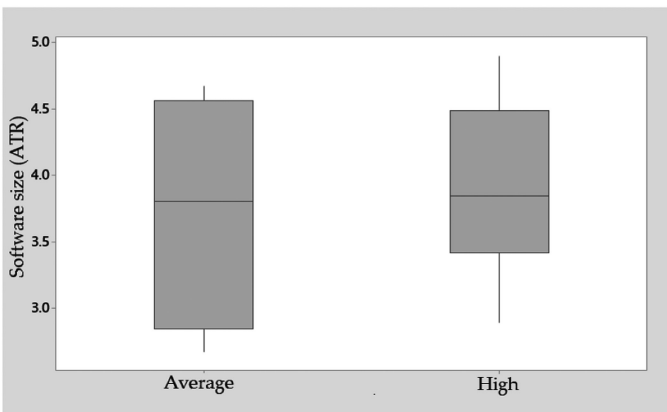


Fig. 1. Software sizes (ATR) of tasks assigned to programmers in both groups.

We normalized the total number of test cases written by programmers in each of the tasks with the respective software size of the task to check the impact of our observation that the average-productivity programmers used more test cases. Fig. 4 shows the data for high-productivity and average-productivity programmers. As the data were not normally distributed (as per the Anderson—Darlington normality test), the Kruskal—Wallis nonparametric test was conducted after removing the outlier. The results showed that the average-productivity programmers wrote more test cases compared with the high-productivity programmers with a statistically significant value of 0.048. The quality of tasks developed by programmers was checked, revealing that all the tasks met and satisfied project quality standards.

The executions are normalized with the software size of the task to check the impact of the number of test case executions. Fig. 5 shows the box plot of this data. As the data were normally distributed (as per the Anderson—Darlington normality test), the two-sample t tests showed that the average-productivity programmers performed more test executions compared with the high-productivity programmers with a statistically significant value of 0.019.

7 Analyzing the Similarity Between Task Processes

In this section, we discuss the results of the second field study, which addressed RQ2 and RQ3. In this study, 3 model-based testing projects were selected, in which a total of 18 programmers were chosen, and each programmer executed 2 tasks that were modeled. A total of 36 tasks were modeled as Markov chains in this study—12 from project 1, 10 from project 2, and 14 from project 3.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

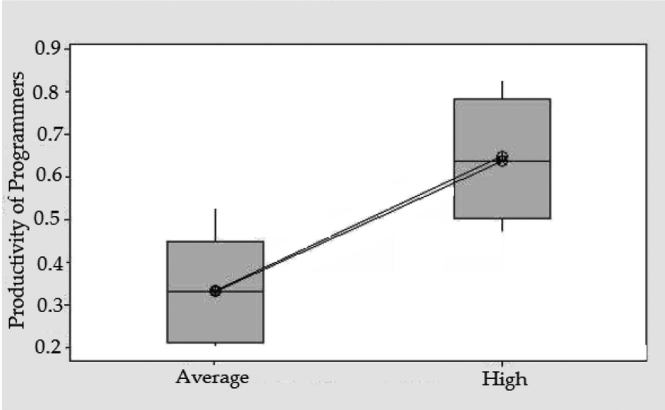


Fig. 2. Productivity of programmers in the two groups.

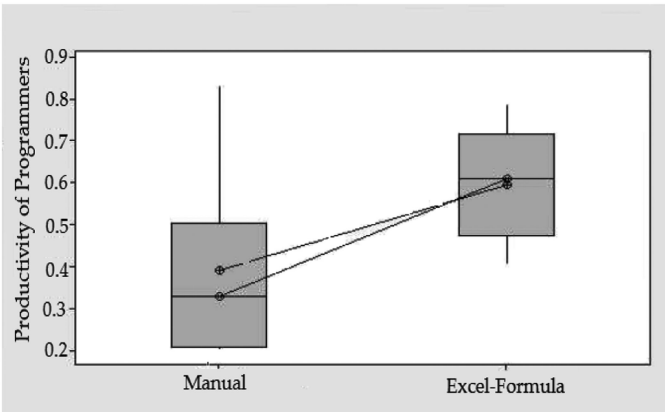


Fig. 3. Productivity of programmers versus automation used by programmers.

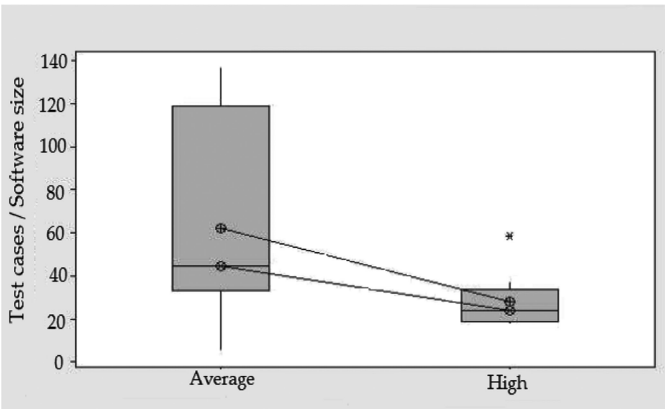


Fig. 4. Number of test cases derived by programmers.

7.1 Modeling the Task Process as Markov Chains

First, we converted the task process as a Markov chain. However, we needed to have the same size and state space for both Markov chains so as to compare two Markov chains. For this, we simply added the missing steps in the task process as dummy steps (which were visited 0 times in the task process). In our study, the total number of distinct steps in the task processes was 27. Hence, the state transition matrix was $27X27$. Further details and examples are given in [66], [67].

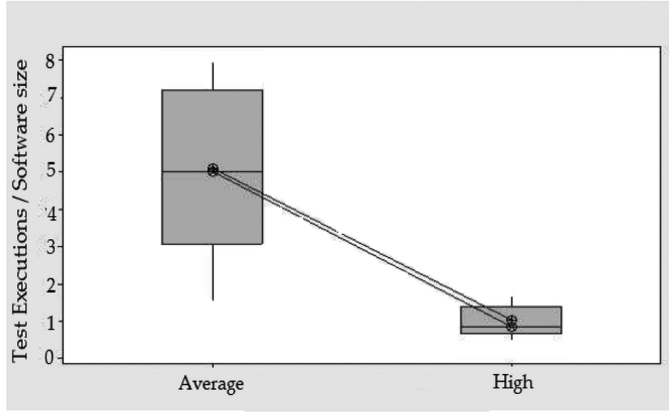


Fig. 5. Number of test executions by programmers.

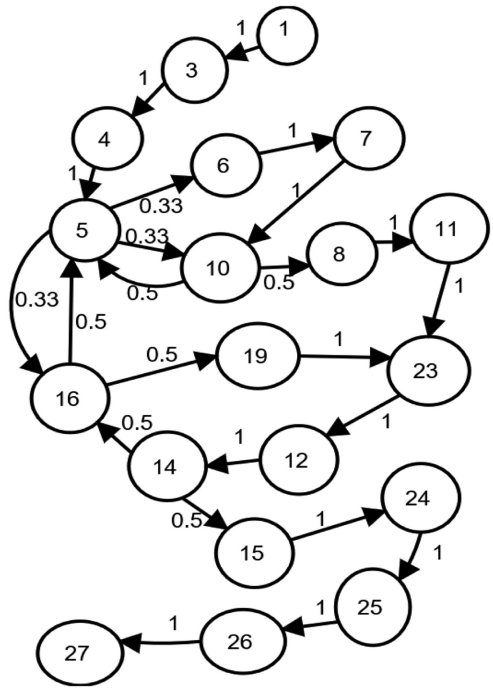


Fig. 6. A sample task process of a high-productivity programmer modeled as a Markov Chain.

We model all the tasks of high- and average-productivity programmers. Figs. 6 and 7 show a sample task processes of high- and average-productivity programmers modeled as Markov chains. The numbered labels in Figs. 6 and 7 represent unique steps in the task processes of all programmers in model-based unit-testing tasks. The names of these 27 steps are mentioned in [66] and include most of the steps in Table 4. (The number of steps in these is different from that in Table 4 — based on discussion with the programmers in this study, some newer steps were identified in the task processes in these projects.)

7.2 Similarity in Task Processes of Programmers (RQ2)

The distances between the task process of a programmer and the task processes of other programmers in the projects are shown using a box plot in Fig. 8. (Therefore, for project 1, the distance between the task processes of a programmer

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

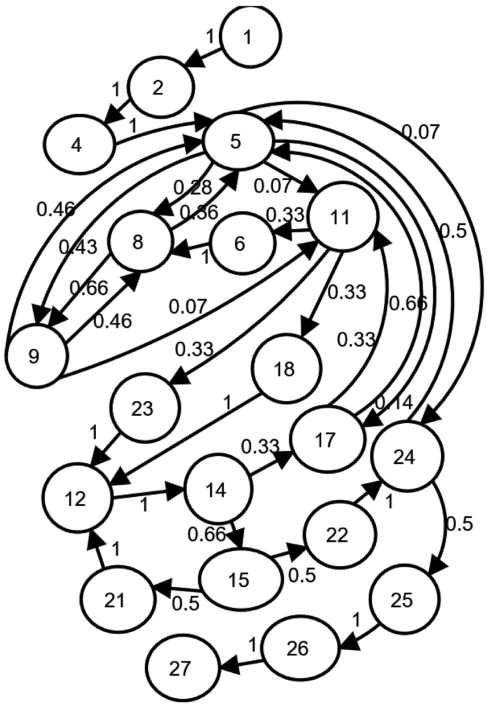


Fig. 7. A sample task process of an average-productivity programmer modeled as Markov Chain.

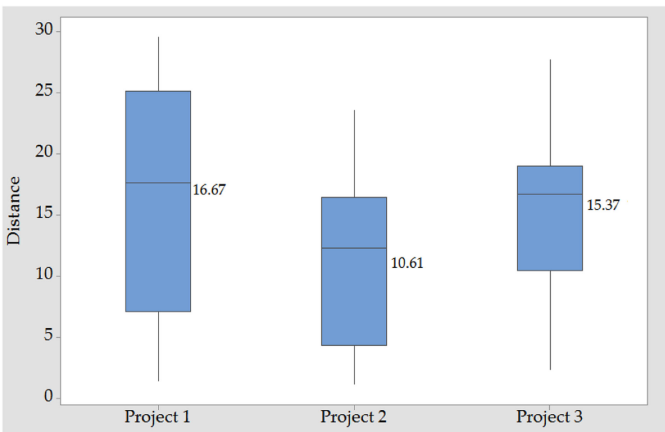


Fig. 8. Distance between the task processes of all programmers.

and the task processes of the other five programmers is shown in the plot for project 1.)

We compute the average distance between the task processes of tasks executed by each individual programmer. This is plotted using a box plot, as shown in Fig. 9. (Box for Project 1 has 6 data points, 1 for each of the 6 programmers; project 2 has 5 data points; and project 3 has 7 data points.) As we can see, these distances are significantly lesser than the distance between the task processes of a programmer and the task processes of other programmers. We can therefore say that *although programmers do not use exactly the same task process for similar tasks, each programmer uses somewhat similar task processes for executing similar tasks, and task processes used by different programmers differ substantially*.

Using the data computed for Fig. 9, we then plotted the average distances for the high- and average-productivity programmers separately—this is shown in Fig. 10. (Therefore,

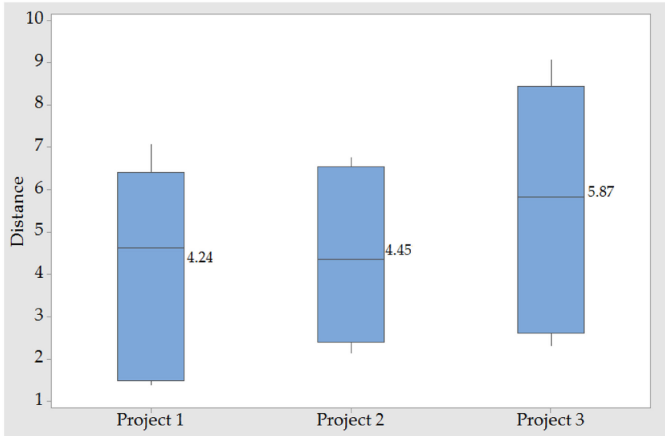


Fig. 9. Distance between the task processes of a programmer.

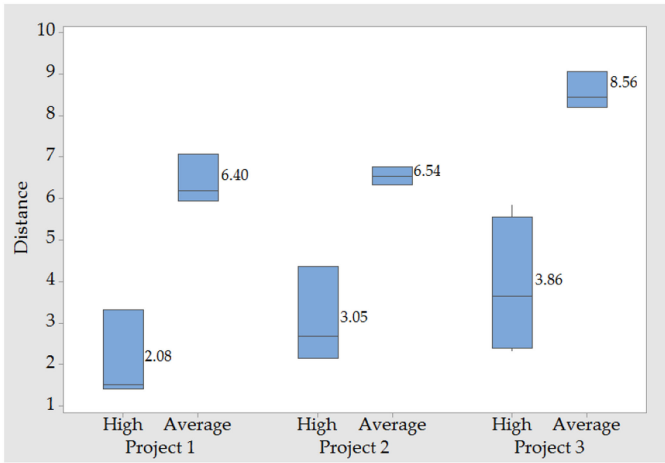


Fig. 10. Distance between the task processes of a programmer in each group.

for project 1, each of the boxes for high and average productivity has 3 data points.) It is clear that the mean/median of distances between the task processes of a high-productivity programmer is significantly lesser than the mean/median of distances between the task processes of an average-productivity programmer. We can therefore say that *the task processes used by a high-productivity programmer for similar tasks are much closer to each other than the task processes used by an average-productivity programmer*. In other words, each high-productivity programmer has evolved a stable task process for this task and uses similar task processes for executing the tasks, while the average-productivity programmer seems to be trying more variations and the task processes for the tasks are less similar. This is an important observation regarding how the two groups of programmers execute tasks.

7.3 Differences Between High- and Average-Productivity Programmers (RQ3)

Fig. 8 shows the distance between the task processes of a programmer and the task processes of other programmers in a project. From these data, we extracted data points about the distance between the tasks of a high-productivity programmer and the tasks of other high-productivity programmers and the distance between task process of an average-productivity programmer and those of other

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

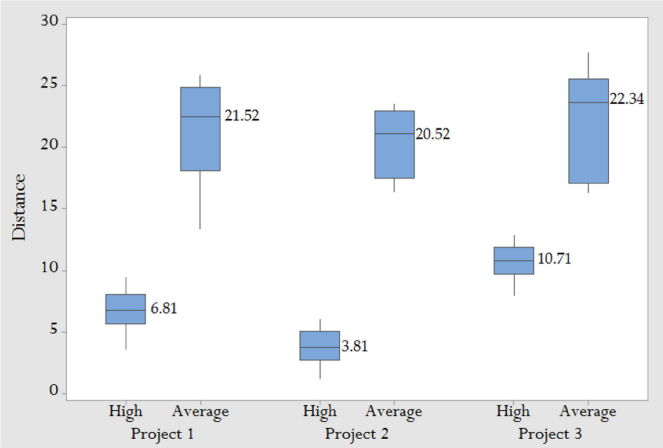


Fig. 11. Distance between the task processes of all programmers in a group.

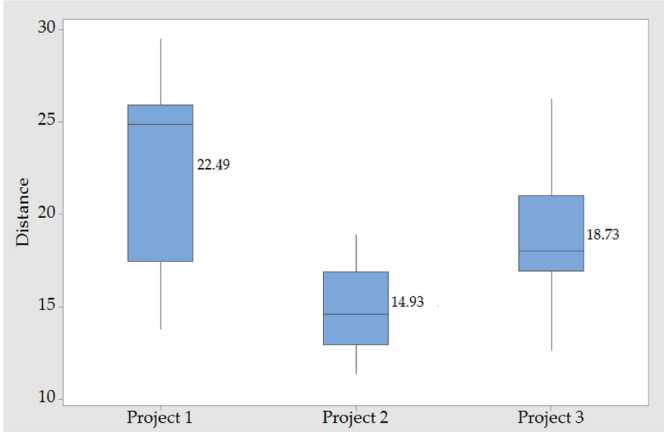


Fig. 12. Distance between the task processes of high- and average- productivity programmers.

average-productivity programmers. This is plotted in Fig. 11. It is clear that the mean/median of distances between the task processes of high-productivity programmers is much less than the mean/median of distances between the task processes of average-productivity programmers. Hence, we can say that *the task processes of programmers in a high-productivity group are more similar to each other, while the task processes of programmers in the average-productive group vary from each other*.

From data for Fig. 8, we also extracted data about the distance between the task processes of high-productivity programmers and the task processes of average-productivity programmers in each project. This is shown using a box plot in Fig. 12. As we can see, the distance between task -processes of average-productivity and high-productivity programmers is quite large, indicating that the task processes of an average-productivity programmer are very different from the task processes of a high-productivity programmer.

8 Impact of Transferring Task Processes of High-Productivity Programmers to Average-Productivity Programmers

As we have seen, the task process used by high-productivity programmers to execute an assigned task is often different than the ones used by average-productivity programmers. A question then arises—"Can average-productivity programmers learn from peers who have become more productive?" This question, in many ways, is the motivation behind the study of task processes—"Can the productivity of average-productivity programmers be improved by improving the task processes they use, and for improving the task process, can these programmers simply follow the task processes used by high-productivity programmers?"

In this section, we report the results of the impact of transferring the task processes of high-productivity programmers to the average-productivity programmers (RQ4). In this field study, we considered three projects and a total of 24 programmers, with each programmer executing at least 2 tasks at the start and end of the study.

Among these projects, the first two were treatment projects, while project 3 was the baseline project. For treatment projects, data were collected about task processes as described earlier. The baseline project was included to provide data on productivity improvement that may take place naturally due to the experience in executing similar tasks over time. Therefore, in the baseline project, we did not inform the project manager and programmers about our approach and used the weekly activity report for determining the effort (as no videos were captured).

All the projects were studied over *a period of about 10 months*. We studied the productivity of high- and average-productivity groups at the start of the project and again after the transfer was done—about 6 months after the programmers were independently executing tasks. (The number of programmers in the two treatment projects reduced by one, as one programmer in the average-productivity group in treatment project 1 and one programmer in the high-productivity group in treatment project 2 left in between.)

8.1 Transferring the Task Process

In the treatment projects, we obtained the task processes of the two groups. We then identified the key differences between the task processes of high- and average-productivity programmers, which we (including the project managers and programmers) believed were the main cause of the productivity difference.

We followed two different approaches in the two projects to transfer the identified differences from the high-productivity programmers to the average-productivity programmers.

In the first one (treatment project 1), we presented the differences in task processes as the best practices observed while executing a task (in a 2-hour session). We prepared two examples of executing a task for each of the identified differences. One example showed the best way of executing a task (taken from a high-productivity programmer's task process), and another example presented a poor way of executing the task (taken from the task processes of an average-productivity programmer). This was followed by an average-productivity programmer executing a couple of tasks together with a high-productivity programmer. (This pairing was done by the project manager informally.)

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

TABLE 5
The Productivity of Each Group in Treatment and Baseline Projects

	Treatment project 1		Treatment project 2		Baseline project	
	High Prod. Group	Avg. Prod. Group	High Prod. Group	Avg. Prod. Group	High Prod. Group	Avg. Prod. Group
Initial Productivity before transfer	2.2	0.8	1.8	0.69	1.9	0.85
Final Productivity after transfer	2.5	1.9	1.95	1.05	1.98	0.98

Some of the topics that we presented to programmers were as follows:

- Creation of test environment for the step "setup test project".
- Using Excel formulas for the step "automation for output variable".
- Selection of signals for understanding the behavior of code for the step "understand code behavior".
- Performing too many test executions by programmers for the step "run test cases".
- Generating many numbers of test cases for the step "write test case inputs".
- Identifying equivalent classes of test input variables for the step "create/modify aspects".
- Identifying only required library classes for the step "load files".
- Avoiding too many switching's between various documents for the step "copy information to Excel".
- Spending time to analyze functional documents at the beginning for the step "analyze functional document".
- Documenting test specification and report for the step "documentation".

In the second treatment project, the transfer was done informally. The project manager was briefed about the study and the differences. He then agreed to share with the team the differences in a suitable manner. No training was conducted for average-productivity programmers.

8.2 Analysis and Results

For all the three projects (the two treatment projects and the baseline project), we determined the productivity of the two groups of programmers at the beginning of the study. We performed a similar exercise 6 months after the transfer of process in the two treatment projects. We computed the productivity of each programmer and took the average for computing the average productivity of a group. Table 5 shows the average productivity (in ATRs per hour) of various groups at the beginning and after the transfer.

As we can see, the productivity of average-productivity programmers improved from 0.8 to 1.9 in treatment project 1, where we transferred the task processes to the average-productivity programmers through training. The average productivity improved from 0.69 to 1.05 in treatment project 2, where we transferred the task processes implicitly through team meetings. Hence, the productivity of average-productivity programmers in the two treatment projects improved by 135 percent and 52 percent respectively. In the baseline project, during the same period, the productivity of average-productivity programmers improved from 0.85 to 0.98, that is, an improvement of 15 percent—this improvement in the productivity of average-productivity programmers probably represents the effect of natural effort that the average-productivity programmers must be putting to improve by learning from team members.

The productivity of high-productivity programmers also improved marginally in all the projects. Further, the improvement was more in the treatment projects where the high-productivity programmers were involved in training the average-productivity programmers when compared with the baseline project.

At the start of this experiment, the ratio of productivity of high-productivity programmers and that of average-productivity programmers was 2.75 and 2.6, respectively, in the treatment projects and 2.25 in the baseline project. After the experiment, this ratio reduced to 1.3 and 1.85, respectively, in the treatment projects, while in the baseline project, this ratio reduced to 2 —a much lesser improvement. In other words, the difference in productivities between high- and average-productivity programmers decreased by 57 percent and 19 percent in the treatment projects, respectively, while in the baseline project it was 5 percent only.

Table 6 shows the individual productivity of average-productivity programmers "before" and "after" in the three projects. As we can see, the productivity of all of them improved. However, the improvement was the largest and most consistent in treatment project 1, where the productivity of all improved more than two times (except programmer 4).

We also studied the similarity between the task processes in the treatment projects. For the treatment projects, we computed the distance between the task processes of high- and average-productivity programmers before starting the study and 6 months after the transfer was conducted.

TABLE 6
Individual Productivity of Average Productivity Programmers

	Treatment project 1		Treatment project 2		Baseline project	
	Before	After	Before	After	Before	After
Programmer 1	1.11	2.47	0.47	0.71	1.05	1.2
Programmer 2	0.91	2.175	0.77	1.14	0.725	0.84
Programmer 3	1.01	2.31	0.82	1.29	0.795	0.9
Programmer 4	0.49	0.97				
Programmer 5	0.83	1.855				
Programmer 6	0.65	1.585				

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

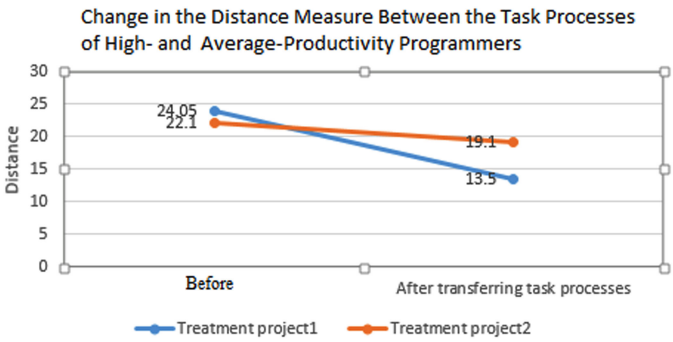


Fig. 13. Change in the distance between the task processes of average- and high-productivity programmers.

The average distance between the task processes of high- and average-productivity programmers in the two treatment projects before and after is shown in Fig. 13. (The average is of the distances between all pairs of tasks of high- and average-productivity programmers.)

If the similarity between the task processes of high-productivity programmers and the task processes of average-productivity programmers increases (i.e., the distance between them decreases), we can safely conclude that the improvement in productivity is due to the improvement in the task process. For similarity between the task processes, as earlier, we derived the state transition matrix for each task process, modeled each task process as a Markov chain, and computed the distance between the state transition matrices as the similarity measure between two task processes.

As we can see, the similarity between the task processes of high- and average-productivity programmers improved substantially—the distance between them reduced by 44 percent and 13 percent, respectively, in the two treatment projects.

The improvement in productivity and similarity with the task processes of high-productivity programmers clearly suggests that the productivity of average-productivity programmers can be improved by improving their task processes. And for improvement, they need to just follow the task processes of those in their peer groups who have high productivity.

The study also suggests that an active approach is much more effective for transferring the processes of high-productivity programmers. Just leaving the programmers to learn from peers through informal interactions is not as effective as a structured and planned transfer through training by peers.

Overall, the study further confirms that the task processes used by the programmers have a strong influence on programmer productivity. Also, the productivity of average-productivity programmers can be improved significantly and brought closer to that of higher-productivity peers by systematically identifying the key differences between how high- and average-productivity programmers execute a task and transferring the processes of high-productivity programmers to average-productivity programmers through planned and properly conducted training.

8.3 Qualitative Feedback

We took limited feedback about this study from the programmers and managers. We conducted interviews with four average-productivity programmers of treatment project 1 (about a year after the training sessions). Table 7 presents the questions that we asked in the interview.

TABLE 7
Interview Questions to Average-Productivity Programmers

Was the training session helpful for improving productivity?
Why did you think your productivity improved? Did you feel the execution of tasks changed due to the learning you had? Which steps did you adopt from the training session? Any feedback that you would like to share?

All the four average-productivity programmers in treatment project 1 found the training session helpful in improving productivity. All the programmers thought that they improved their productivity. Three out of four average-productivity programmers felt that they were now executing their tasks quickly with ease compared with their earlier performance. Although the fourth programmer also felt the same, he opined that further improvement in some steps could boost his productivity.

For our question "which steps did you adopt from the training session?," all the four programmers told us that they are now using all the missing steps. (We also checked the task processes of these programmers after the training and found that they are using the missing steps.)

We could not get in touch with the average-productivity programmers from treatment project 2. However, we asked the project manager of the project about any feedback that he may have received from his team members. The project manager felt that the study helped identify areas of improvement for the productivity of his team members.

9 Limitations and Threats to Validity

Our framework for studying the impact of task processes requires grouping of programmers into two groups of high and average productivity. Consequently, the framework requires a reasonable number of programmers executing a particular type of task for grouping them into two groups of high and average productivity. Hence, this framework can be applied only to tasks such as coding, enhancements, and testing, where more programmers are working on a project. It may not be suitable for tasks in which only a few people are involved, for example, architecture design, project planning, and many others. It should, however, be noted that, for productivity gains, it is generally advantageous to focus on tasks where a larger number of people are involved.

The framework is based on learning by average-productivity programmers from their high-productivity peers. This implicitly assumes that the population on which this framework is applied is such that they are open to learning from peers and may be looking for ways to improve their task execution processes. We feel that, due to this, the framework may be more suitable for junior programmers compared with the experienced senior programmers because the latter group is generally rather small and senior programmers are often in a state where they may have evolved mature personal task processes that may not be amenable to change, or which may have already incorporated learning from peers.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

In our study, we applied this framework on programmers having 1–2 years of experience as advised by the management (as the programmers having 1–3 years of experience are the major workforce in the organization). Typically, the size of a project team in the organization was around 12–14 members, and the number of programmers having less than three years of experience was around 8. We applied this framework to only model-based unit-testing tasks. It was a great deal of effort to get a buy-in for doing this study. We tried to get the buy-in from the management for applying this to other types of tasks, but that was not approved. Consequently, the results of the experiment were clearly not generalizable to other tasks executed by a large number of (junior) programmers, such as coding, bug fixing, and enhancements. Further, the steps that we presented to programmers were also quite specific to one company. We hope that other organizations or groups will use the framework to try it on other tasks to better understand the utility of this approach.

Another key limitation of this work is that the experiments were done in one company only and in one country. It is desirable that more experiments should be conducted across companies, perhaps across countries also, to explore whether studying task processes in the manner described in this work and then transferring them to average-productivity programmers can provide productivity benefits. It should be added that some companies were approached for trying this, but they did not show interest in using the framework being used by one company in theirs.

There can be other qualitative and quantitative variables on each step in a task process. We did not put forward any general framework for identifying and capturing these variables. This is another limitation of this study.

Conducting controlled experiments on live projects is challenging. To repeat the execution of same tasks by other programmers to create a baseline for comparison is generally not possible in companies executing live projects because it has a direct impact on cost and schedule. Due to this, we used only live projects instead of attempting any controlled experiments.

The tasks in the study were similar to each other, though not exactly the same. Further, the tasks were distributed across the two groups almost randomly. Due to this, we felt that no systematic bias was introduced while assigning tasks to the programmers. Additionally, the tasks considered in this study belonged to the same application, same domain, and same underlying platform, and used the same tools for execution.

Similarly, although the projects in this study were not exactly the same, they were similar in terms of overall process, tools, infrastructure, functionality of software, types of tasks, and experience of programmers. Therefore, it is expected that bias due to projects is also minimal in the study.

10 Conclusions

Some programmers are much more productive compared with others, and the productivity difference can be two to three times within programmers with similar background and experience. In this work, we studied the impact of task processes used by programmers to execute the tasks assigned to them on programmer productivity. A task is a project activity assigned to one programmer, and a task process is the sequence of steps the programmer performs to complete the assigned task. For studying the impact of task processes on productivity, we studied some live projects in Robert Bosch Engineering and Business Solutions Limited, a CMMI Level 5 company. We took a few similar model-based testing projects, and in each project, we identified two groups of programmers—high-productivity programmers and average-productivity programmers. We focused on unit-testing tasks for this work.

For studying the task processes of programmers, we captured their computer monitor and then analyzed the task videos to extract the task process used for a task. For quantifying the differences between task processes, we modeled each task process as a Markov chain and measured the difference between task processes as the distance between the state transition matrices of task processes.

Our study showed clearly identifiable differences between the task processes of high-productivity programmers and the task processes of average-productivity programmers. We also showed that these differences in task processes resulted in productivity differences between the two groups of programmers.

We also showed that programmers used similar task processes for executing the same task, but the task processes of different programmers for the same task differed substantially. Also, the task processes of high-productivity programmers were more similar to each other, while the task processes of average-productivity programmers varied more. We showed that the task processes of high-productivity programmers were quite different from the task processes of average-productivity programmers. For transferring the task processes of high-productivity programmers to average-productivity programmers, we identified the important steps that needed to be changed, added, or deleted, and then trained the average-productivity programmer(s) on those. We found that the productivity of average-productivity programmers increased significantly with this and that the similarity between the task processes of high- and average-productivity programmers also increased.

The framework used for assessing the similarity in task processes can also be used in other ways. For example, in a study, we found that the training provided to junior programmers had an effect on their task process as they became similar to the process of the trainer. This has some obvious impact on the selection of trainers for new trainees [66].

Clearly, much more needs be done for better understanding the impact of task processes. For example, the impact of task processes on other commonly assigned tasks to programmers in which many programmers spend a considerable amount of efforts, such as modeling, coding, bug fixing, and enhancements, can be studied. Further, such studies, or even the study we performed, can be conducted in other organizations to see whether similar benefits are obtained. In this work, we focused on identifying possible improvement opportunities for average-productivity programmers—it should be possible to explore the approach for productivity improvement for high-productivity

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

programmers as well. The approach can also be used to understand the task processes of experienced programmers compared with those of less experienced programmers. Many other such possibilities exist.

Developing lightweight methods is also needed, particularly for organizations that may want to apply the framework of improving task processes—the approaches used in this work are quite expensive. For example, for similarity, instead of using Markov chains, simpler and more informal methods may be suitable, particularly for applying the techniques. Similarly, task processes can be documented through simpler means than the ones used in this work.

Acknowledgments

We would like to thank the senior management of Robert Bosch Engineering and Business Solutions Private Limited for allowing us to conduct this study. A special thanks to the project managers and programmers for participating in this study.

References

[1] R. Premraj, M. Shepperd, B. Kitchenham, and P. Forselius, "An empirical analysis of software productivity over time," in *Proc. 11th IEEE Int. Softw. Metrics Symp.*, 2005, pp. 10–20.

[2] D. Galin and M. Avrahami, "Are CMM program investments beneficial? Analyzing past studies," *IEEE Softw.*, vol. 23, no. 6, pp. 81–87, Nov./Dec. 2006.

[3] V. Jovanovic and D. Shoemaker, "ISO 9001 standard and software quality improvement," *Benchmarking Quality Manage. Technol.*, vol. 4, no. 2, pp. 148–159, 1997.

[4] T. K. Varkoi and T. K. Mkinen, "Case study of CMM and spice comparison in software process assessment," in *Proc. Pioneering New Technol.: Manage. Issues Challenges 3rd Millennium*, 1998, pp. 477–482.

[5] P. Jalote, *CMM in Practice: Processes for Executing Software Projects at Infosys*. Reading, MA, USA: Addison-Wesley, 2000.

[6] P. Jalote and G. Jain, "Assigning tasks in a 24-h software development model," *J. Syst. Softw.*, vol. 79, no. 7, pp. 904–911, 2006.

[7] E. E. Grant and H. Sackman, "An exploratory investigation of programmer performance under on-line and off-line conditions," *IEEE Trans. Human Factors Electron.*, vol. HFE-8, no. 1, pp. 33–48, Mar. 1967.

[8] L. Prechelt, "An empirical study of working speed differences between software engineers for various kinds of task," *IEEE Trans. Softw. Eng.*, vol. 33, no. 10, pp. 23–29, Oct. 2000.

[9] L. Macaulay, "The importance of human factors in planning the requirements capture stage of project," *Int. J. Project Manage.*, vol. 15, no. 1, pp. 39–53, 1997.

[10] T. DeMarco and T. Lister, *Peopleware: Productive Projects and Teams*. New York, NY, USA: Dorset House Publishing, 1987.

[11] W. D. Brooks, "Software technology payoff: Some statistical evidence," *J. Syst. Softw.*, vol. 2, no. 1, pp. 3–9, 1981.

[12] R. H. Rasch, "An investigation of factors that impact behavioural outcomes of software engineers," in *Proc. Conf. SIGCPR*, 1991, pp. 38–53.

[13] B. Kitchenham and E. Mendes, "Software productivity measurement using multiple size measures," *IEEE Trans. Softw. Eng.*, vol. 30, no. 12, pp. 1023–1035, Dec. 2004.

[14] R. Berntsson-Svensson and A. Aurum, "Successful software project and products: An empirical investigation," in *Proc. ACM/IEEE Int. Symp. Empirical Softw. Eng.*, 2006, pp. 144–153.

[15] M. K. Goncalves, L. R. de Souza, and V. M. Gonzalez, "Collaboration, information seeking and communication: An observational study of software developers work practices," *J. Universal Comput. Sci.*, vol. 17, no. 14, pp. 1913–1930, 2011.

[16] R. H. Rasch and H. L. Tosi, "Factors affecting software developers' performance: An integrated approach," *Manage. Inf. Syst. Res. Center*, vol. 16, no. 3, pp. 395–413, 1992.

[17] T. Bruckhaus, N. Madhavii, I. Janssen, and J. Henshaw, "The impact of tools on software productivity," *IEEE Softw.*, vol. 13, no. 5, pp. 29–38, Sep. 1996.

[18] J. D. Blackburn, G. D. Scudder, and L. N. V. Wassenhove, "Improving speed and productivity of software development: A global survey of software developers," *IEEE Trans. Softw. Eng.*, vol. 22, no. 12, pp. 875–885, Dec. 1996.

[19] B. P. Bailey, J. A. Konstan, and J. V. Carlis, "The effects of interruptions on task performance, annoyance, and anxiety in the user interface," in *Proc. Int. Conf. Human-Comput. Interaction*, 2011, pp. 593–601.

[20] T. McGibbon, "*A business case for software process improvement revised*," DoD Data center for software, Griffiss AFB, NY: DACS, Sep. 1999.

[21] T. Tan, Q. Li, B. Boehm, Y. Yang, M. He, and R. Moazeni, "Productivity trends in incremental and iterative software development," in *Proc. 3rd Int. Symp. Empirical Softw. Eng. Meas.*, 2009, pp. 1–10.

[22] W. S. Humphrey, "Using a defined and measured personal software process," *IEEE Softw.*, vol. 13, no. 3, pp. 77–88, May 1996.

[23] M. Morisio, "Applying the PSP in industry," *IEEE Softw.*, vol. 17, no. 6, pp. 90–95, Nov./Dec. 2000.

[24] J. Kamatar and W. Hayes, "An experience report on the personal software process," *IEEE Softw.*, vol. 17, no. 6, pp. 85–89, Nov./Dec. 2000.

[25] P. M. Johnson, H. Kou, J. M. Agustin, C. Chan, C. A. Moore, J. Miglani, S. Zhen, and W. E. Doane, "Beyond the personal software process: Metrics collection and analysis for the differently disciplined," in *Proc. Int. Conf. Softw. Eng.*, 2003, pp. 641–646.

[26] G. Canfora, A. Cimitile, and C. Aaron, "Lessons learned about distributed pair programming: What are the knowledge needs to address," in *Proc. 12th IEEE Int. Workshop Enabling Technol.: Infrastructure Collaborative Enterprises*, 2003, pp. 314–319.

[27] S. Pietinen, V. Tenhunen, and M. Tukiainen, "Productivity of pair programming in a distributed environment – results from two controlled case studies," in *Proc. Eur. Conf. Softw. Process Improvement*, 2008, pp. 47–58.

[28] M. Ciolkowski, O. Laitenberger, D. Rombach, F. Shull, and D. Perry, "Software inspections, reviews and walkthroughs," in *Proc. 24th Int. Conf. Softw. Eng.*, 2002, pp. 641–642.

[29] M. V. Mantyla and C. Lassenius, "What types of defects are really discovered in code reviews?" *IEEE Trans. Softw. Eng.*, vol. 35, no. 3, pp. 430–448, May/Jun. 2009.

[30] G. J. Myers, "A controlled experiment in program testing and code walkthroughs/inspections," *Commun. ACM*, vol. 21, no. 9, pp. 760–768, 1978.

[31] G. K. Gill and C. F. Kemerer, "Quantitative and qualitative differences between experts and novices in chunking computer software knowledge," *Int. J. Human.-Comput. Interaction*, vol. 6, no. 1, pp. 105–118, 1994.

[32] L. Gugerty and G. Olson, "Debugging by skilled and novice programmers," in *Proc. SIGCHI Conf. Human Factors Comput. Syst.*, 1986, pp. 171–174.

[33] A. N. Meyer, T. Fritz, G. C. Murphy, and T. Zimmermann, "Software developers perceptions of productivity," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, 2014, pp. 19–29.

[34] R. C. Martin and R. S. Koss, *Engineering notebook: An extreme programming episode. Advanced principles, patterns, and process of software development*, Prentice Hall, 2001.

[35] B. Hartmann, D. MacDougall, J. Brandt, and S. R. Klemmer, "What would other programmers do: Suggesting solutions to error messages," in *Proc. SIGCHI Conf. Human Factors Comput. Syst.*, 2010, pp. 1019–1028.

[36] M. P. Robillard, W. Coelho, and G. C. Murphy, "How effective developers investigate source code: An exploratory study," *IEEE Trans. Softw. Eng.*, vol. 30, no. 12, pp. 889–903, Dec. 2004.

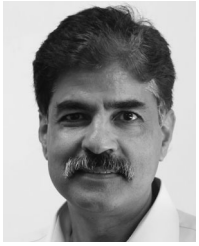
[37] R. W. Bowdidge and W. G. Griswold, "How software engineering tools organize programmer behavior during the task of data," *Empirical Softw. Eng.*, vol. 2, pp. 221–267, 1997.

[38] J. Lawrance, C. Bogart, M. Burnett, R. Bellamy, K. Rector, and S. D. Fleming, "How programmers debug, revisited: An information foraging theory perspective," *IEEE Trans. Softw. Eng.*, vol. 39, no. 2, pp. 197–215, Feb. 2013.

[39] A. D. Grip and J. Sauermann, *Scientific Management*. Evanston, IL, USA: Routledge, 2004.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.

[40] A. Grip and J. Sauermann, *Taylorism Transformed: Scientific Management Theory Since 1945*. Chapel Hill, NC, USA: Univ. North Carolina Press, 2016.
[41] S. Vegas, Ó. Dieste, and N. Juristo, "Difficulties in running experiments in the software industry: Experiences from the trenches," in *Proc. IEEE/ACM 3rd Int. Workshop Conducting Empirical Stud. Ind.*, 2015, pp. 3–9.
[42] D. I. K. Sjoberg, J. E. Hannay, O. Hansen, V. Kampenes, A. Karahansanovic, N. Kristian, and A. C. Rekdal, "A survey of controlled experiements in software engineering," *IEEE Trans. Softw. Eng.*, vol. 31, no. 9, pp. 733–753, Sep. 2005.
[43] D. S. W. Yu and S. Huang, "Software productivity measurement," *AT& T Tech. J.*, vol. 69, no. 1, pp. 110–120, 1990.
[44] W. Scacchi and D. Hurley, "Understanding software productivity," *Softw. Eng. Knowl. Eng.: Trends Next Decade*, vol. 4, pp. 273–326, 1995.
[45] D. Anselmo and H. F. Ledgard, "Measuring productivity in the software industry," *Commun. ACM*, vol. 46, no. 11, pp. 121–125, 1995.
[46] C. Jones, "Software metrics: Good, bad and missing," *Comput.*, vol. 27, no. 9, pp. 98–100, 1994.
[47] P. Devanbu, S. Karstu, W. Melo, and W. Thomas, "Analytical and empirical evaluation of software reuse metrics," in *Proc. 18th Int. Conf. Softw. Eng.*, 1996, pp. 189–199.
[48] M. Zhou and A. Mockus, "Developer fluency: Achieving true mastery in software projects," in *Proc. 18th ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, 2010, pp. 137–146.
[49] A. Mockus, R. T. Fielding, and J. D. Herbsleb, "Two case studies of open source software development: Apache and Mozilla," *ACM Trans. Softw. Eng. Methodology*, vol. 11, no. 3, pp. 309–346, 2002.
[50] M. Cohn, "Estimating with use case points," *Methods Tools*, vol. 13, no. 3, pp. 3–13, 2006.
[51] P. B. Wilson, "Sizing software with testable requirements," *Syst. Develop. Manage.*, vol. 1, pp. 10–34, 2000.
[52] Snag-It. 2014. [Online]. Available: https://www.techsmith.com/snagit.html
[53] H. K. Meena, I. Saha, K. K. Mondal, and T. V. Prabhakar, "An approach to workflow modeling and analysis," in *Proc. OOPSLA Workshop Eclipse Technol. Exchange*, 2005, pp. 85–89.
[54] A. Raj, A. Agrawal, and T. V. Prabhakar, "Transformation of business processes into UML models: An SBVR approach," *Int. J. Sci. Eng. Res.*, vol. 4, no. 3, pp. 647–661, 2013.
[55] B. Curtis, M. I. Kellner, and J. Over, "Process modeling," *Commun. ACM*, vol. 35, no. 9, pp. 75–90, 1992.
[56] O. Leon, "Software processes are software too," in *Proc. 9th Int. Conf. Softw. Eng.*, 1987, pp. 2–13.
[57] M. Suzuki and T. Katayama, "Meta-Operations in the process model HFSP for the dynamics and flexibility of software processes," in *Proc. 1st Int. Conf. Softw. Process*, 1991, pp. 202–207.
[58] L. G. Williams, "Software process modeling: A behavioural approach," in *Proc. 10th Int. Conf. Softw. Eng.*, 1988, pp. 174–186.
[59] L. Williams, "A behavioural approach to software process modelling," in *Proc. 4th Int. Softw. Process Workshop Representing Enacting Softw. Process*, 1988, pp. 167–170.
[60] K. Huff and V. Lessor, "Software process model evolution in the spade environment," *IEEE Trans. Softw. Eng.*, vol. 19, no. 12, pp. 1128–1144, Dec. 1993.
[61] K. E. Huff and V. R. Lessor, "A planbased intelligent assistant that supports the software development process," in *Proc. 3rd Softw. Eng. Symp. Practical Softw. Develop. Environ.*, 1989, pp. 97–106.
[62] L. Wen, W. van der Aalst, J. Wang, and J. Sun, "Mining process models with non-free-choice constructs," *Data Mining Knowl. Discovery*, vol. 15, no. 2, pp. 145–180, 2007.
[63] B. Sericola, *Markov Chains: Theory and Applications*. Hoboken, NJ, USA: Wiley, 2013.
[64] A. Charu, A. Hinneburg, and D. A. Keim, "On the surprising behavior of distance metrics in high dimensional spaces," in *Proc. Int. Conf. Database Theory*, 2001, pp. 420–434.
[65] K. Damodaram and J. Pankaj, "Effect of task processes on programmer productivity in model-based testing," in *Proc. India Softw. Eng. Conf.*, 2013, pp. 23–28.
[66] K. Damodaram, PhD thesis, 2017. [Online]. Available: https://repository.iiitd.edu.in/xmlui/handle/123456789/513
[67] K. Damodaram and J. Pankaj, "High productivity programmers use effective task processes in unit-testing," in *Proc. Asia-Pacific Softw. Eng. Conf.*, 2015, pp. 32–39.



Pankaj Jalote received the BTech degree from IIT Kanpur, the MS degree from Pennsylvania State University, and the PhD degree from the University of Illinois at Urbana-Champaign. He is a distinguished professor with IIIT-Delhi (Indraprastha Institute of Information Technology Delhi). He was the founding director of IIIT-Delhi from 2008 to 2018. Prior to this, he has been a chair professor with IIT Delhi, professor with the Department of Computer Science and Engineering, Indian Institute of Technology Kanpur, India, where he was also the head of the Department, and an assistant professor with the University of Maryland at College Park. From 1996 to 1998, he was vice president with Infosys Technologies Ltd., and from 2003 to 2004, was a visiting researcher with Microsoft Corporation, Redmond. He is the author of four books including the highly acclaimed *CMM in Practice*, which has been translated in Chinese, Japanese, Korean etc, and the best selling text book *An Integrated Approach to Software Engineering*. His main area of interest is software engineering and now higher education. He has served on the editorial boards of the *IEEE Transactions on Software Engineering*, the *International Journal on Empirical Software Engineering*, and the *IEEE Transactions on Services Computing*. He is a fellow of the IEEE and INAE.

Damodaram Kamma received the PhD degree in software engineering from IIIT Delhi, India, in 2017. He is currently working as a manager and handles software productivity management for embedded solutions at Robert Bosch Engineering and Business Solutions Limited, Bangalore, India. In this position, he is responsible for achieving organizational software-productivity goal by defining and analyzing productivity metrics along with identifying initiatives for improving software productivity. His research interests include empirical studies on programmers, software teams and process improvements for improving programmer, and software productivity.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: Pontifica Universidade Catolica do Parana. Downloaded on April 12,2026 at 23:30:04 UTC from IEEE Xplore. Restrictions apply.